Class 20: Introduction to Software Engineering for Data Products

Session 20 of 36 – Opening the Software Engineering and MLOps Block

Data-driven Systems Engineering

Agenda

1. **Software engineering and ML**
   From programming to engineering
2. **Requirements and quality**
   Attributes, functional and non-functional
3. **Process and system types**
   Four activities, two running cases
4. **Design**
   Modularity, coupling, interfaces
5. **Quality assurance**
   Testing code, data, and models
6. **Evolution and MLOps**
   Maintenance and DevOps

1 Software engineering and ML

IN THIS PART

- From programming to software engineering
- A model is one box in a system

From programming to software engineering

Software engineering is programming integrated over time.

Winters, Manshreck, and Wright, *Software Engineering at Google*, 2020

Time
how long it lives

Maintained, upgraded, and eventually replaced long after it was written.

Scale
size and reach

More users, data, and dependencies, and more ways to fail.

Trade-offs
hard choices

Cost, risk, and deadlines force choices that someone must later explain.

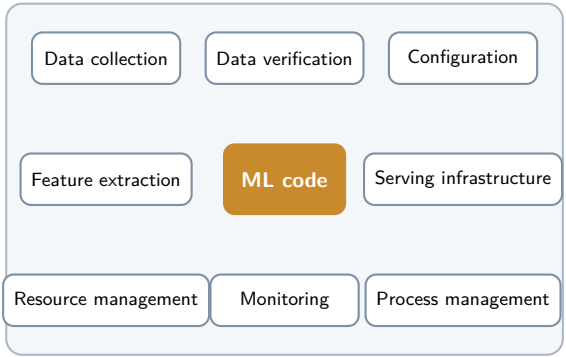
Data
added by ML

The system changes without a code change: data shifts and models are retrained.

Goal: dependable software, at acceptable cost, that can keep changing.

NEW **Trade-off:** gaining one quality by giving up another, e.g. speed for cost. **Dependable:** behaving correctly when it matters, without causing harm.

A model is one box in a system



The model is the small part; the rest decides whether it works in practice.

Redrawn after Sculley et al., NeurIPS 2015; schematic, not to scale.

NEW **Serving:** making a trained model answer requests. **Monitoring:** watching a running system and its data.

2 Requirements and quality

Previously: the model is the small part; the system around it decides whether it works.

IN THIS PART

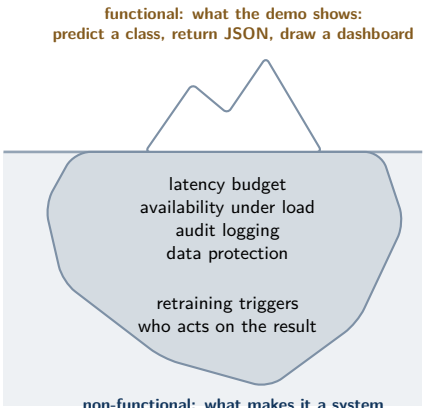
- Quality attributes and their KPIs
- Requirements come before implementation
- Functional and non-functional requirements

Quality attributes of ML-enabled software

ATTRIBUTE	WHY IT MATTERS	KPIs / METRICS	WHERE IT RETURNS IN MLOPS
Maintainability	Models, features, and infrastructure all change.	**Modules touched per change; test coverage**	Modular pipelines and automated tests.
Dependability	Wrong predictions can cause harm.	**Availability; error rate; recall on fraud**	Monitoring, alerts, and rollback.
Efficiency	Latency and compute cost decide feasibility.	**p95 latency; requests per second; cost per 1,000 predictions**	Serving, batching, and scaling.
Usability	Operators must understand outputs.	**Task success rate; time to decision**	Decision-support interfaces.
Traceability	Decisions must be auditable afterward.	**Share of predictions logged with model version**	Versioning and audit logs.

NEW **KPI:** a measurable indicator of a goal. **Latency (p95):** time from request to response; p95 means 95% of requests are faster. **Availability:** share of time the system works. **Traceability:** following a decision back to its sources. **Rollback:** going back to the previous version.

Requirements come before implementation



- Software engineering starts by clarifying what the system must do and how well.
- A strong model does not rescue unclear goals or vague success criteria.
- Many ML failures are non-functional: the model exists, but the service is slow, brittle, or opaque.
- In data products, requirements cover software behavior and data behavior.

NEW **Functional requirement:** what the system does. **Non-functional requirement:** how well it does it. **Service:** a program that keeps running and answers other programs through an agreed interface. **API:** the set of operations a service offers.

Functional and non-functional requirements

F **Functional:** what the system does

NF **Non-functional:** how well it does it

Type	Requirement (example)	Quality attribute	How we check it
F	Return a fraud probability and decision.	–	Input/output tests
F	Show live sensor readings on a dashboard.	–	Acceptance check
NF	Swap the model without changing the API.	Maintainability	Contract tests on swap
NF	Reject invalid input with HTTP 400.	Dependability	Invalid-input tests
NF	95% of requests answered within 800 ms.	Efficiency	Load test; p95 latency
NF	Show the probability and threshold.	Usability	Review with operators
NF	Log each prediction and model version.	Traceability	Inspect the log

Non-functional requirements are quality attributes made measurable.

NEW **Acceptance criterion:** a measurable condition that tells us a requirement is met. **Load test:** sending many requests at once to see how the system behaves. **Contract test:** checks that a service still honors its API.

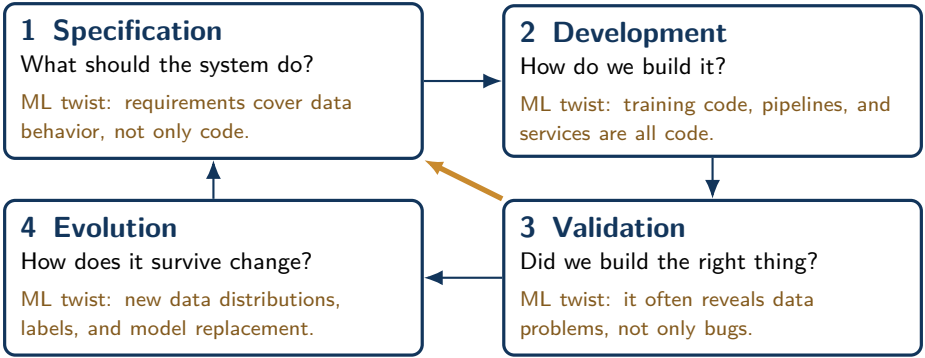
3 Process and system types

Previously: quality attributes become measurable requirements.

IN THIS PART

- Four activities, one loop
- Kinds of software systems
- A data collection system: the pasteurization stream

Four activities, one loop



The gold arrow: validation often sends us back to the specification, because data problems show up as requirement problems.

NEW **Validation:** checking that we built the right thing (verification: that we built it right). **Label:** the true outcome of a case, e.g. fraud or not. **Data distribution:** how values are spread; it can shift over time.

Kinds of software systems

Stand-alone application
A Streamlit dashboard running on one laptop

Transaction-based system
Fraud scoring API: request in, answer out
FRAUD CASE

Embedded control system
The pasteurizer's own controller (outside our scope)

Batch processing system
Scheduled retraining (Class 27)

Data collection system
Sensor stream feeding an online predictor
PASTEURIZATION CASE

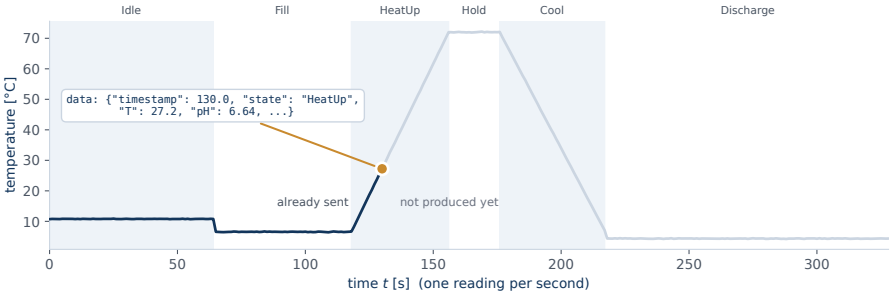
System of systems
Model service + feature pipeline + CRM + alert routing

Generic product: the vendor owns and evolves it (e.g. scikit-learn).

Customized product: scoped and changed with the client (e.g. a plant dashboard).

NEW **Transaction-based:** a request in, an answer out. **Batch:** many items processed together, on a schedule. **Embedded:** software inside a device. **System of systems:** independent systems that cooperate. **CRM:** customer-management software.

A data collection system: the pasteurization stream



- The server pushes one event per reading: a continuous flow, not a request and a reply.
- Its quality attributes differ from the fraud API: timeliness and continuity matter more than the latency of one call.

NEW **Stream:** data pushed continuously, one event at a time. **Event:** one message, here a sensor reading with its timestamp. **Server-Sent Events (SSE):** a web protocol for a server to push events to a client. **Timeliness, continuity:** readings arrive on time and without gaps.

4 Design

Previously: we know what to build; now, how to structure it.

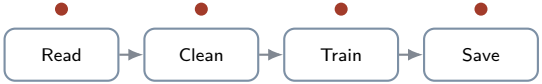
IN THIS PART

- Four parallel principles: modularity, high cohesion, low coupling, interface as contract
- Change scenarios: who has to change?
- Example: a prediction service as a pipeline of responsibilities

Modularity: decompose by what is likely to change

PRINCIPLE 1 OF 4

By processing step



A change in the data format touches all four steps.

By design decision (information hiding)



The same change touches one module.

- A **module** has a responsibility, a public interface, and hidden internals.
- **Information hiding** (Parnas, 1972): each module hides one design decision likely to change.
- **Benefits:** independent work, testing alone, replacement, reuse.
- **Cost:** interfaces take effort; many tiny modules are as bad as one big one.

NEW **Module:** a part with one clear responsibility and a boundary. **Interface:** how other parts use a module. **Information hiding:** a module keeps one design decision private, so changing it does not ripple out.

Modularity in practice: the wine project (Class 19)

One file per responsibility; each one hides a decision that is likely to change.

Module	Responsibility	Decision it hides	A change that stays inside
`configs/*.yaml`	The settings of each run	Hyperparameters and search space	Try new values without editing code
`train.py`	Fit a model, save the artifact	Model family, training loop	Swap the model, keep the rest
`sweep.py`	Run a hyperparameter sweep	Search strategy	Switch the search method
`evaluate.py`	Evaluate once on the test set	Metrics, test protocol	Add a metric
`register.py`	Promote an artifact to the registry	Promotion rules	Tighten the release gate
`figures.py`	Draw the report figures	Plot style and layout	Restyle without retraining

Source: the `wine-wandb` project, Class 19 (repository connections).

NEW **Artifact:** a file produced by training and deployed, e.g. a model file. **Hyperparameter:** a setting chosen before training. **Sweep:** a systematic search over hyperparameter values. **Model registry:** a catalog of versioned models, with promotion rules.

High cohesion: one module, one reason to change


```
do_everything()
    load data
    compute features
    train
    evaluate
    serve
```

five reasons to change in one place

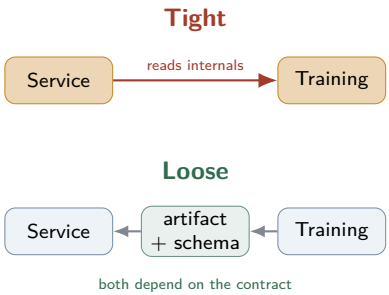
Responsibility	Changes when. . .
Load data	the source or the format changes
Compute features	a new signal or definition is needed
Train	the algorithm or its settings change
Evaluate	the metric or the test protocol changes
Serve	the API contract changes

Heuristic: if you need the word "and" to describe a module, split it.

NEW **Cohesion:** how closely the parts of a module belong together. **Responsibility:** one job a module is in charge of.

Low coupling: depend on interfaces, not on internals

PRINCIPLE 3 OF 4



Form	Example in ML	Loosen by
Content	A dashboard reads private attributes of the model object	Expose a method or a response field
Environment	A pickled model loads only with the library version that wrote it	Pin versions; rebuild with the environment
Schema	Training and serving each assume the same column order	Define the schema once; validate at the boundary
Temporal	Step B only works if step A ran earlier in the session	Explicit inputs and outputs; a pipeline that runs in order

NEW **Coupling:** how much a module depends on another's internals. **Schema:** the declared structure of data: names, types, order. **Pickle:** Python's format for saving objects; load only trusted files. **Session:** one run of a notebook or program. **Pin versions:** fix exact library versions.

Interface as contract: what to specify

PRINCIPLE 4 OF 4

An interface is a contract: inputs, outputs, and behavior on error are agreed in advance.

Element	Fraud API (request and response)	Pasteurization stream (events)
Input	A `features` list of 30 finite numbers in a fixed order.	None: the client opens the stream and listens.
Output	`fraud_probability`, `threshold`, `is_fraud`, `model_version`, `request_id`.	One JSON event per reading: `timestamp`, `state`, `T`, `pH`, `Q_in`, ...
Errors	HTTP 400 with a message; never a silent answer.	The connection closes; the client decides whether to reconnect.
Guarantees	The same input and artifact give the same probability.	One reading per second, in time order.
Versioning	`model_version` in every response.	No version field: a column change breaks consumers silently.

Compatible changes
Add a response field or an optional input.

Breaking changes
Rename, reorder, or remove a field; change units or meaning.

NEW **HTTP status:** 200 = OK, 400 = invalid request, 500 = server failure. **Deterministic:** the same input always gives the same output. **Model version:** which artifact produced a response.

Change scenarios: who has to change?

	Data	Training	Artifact	Service	Monitor	Modular	Entangled
The data source is replaced	●				○	1/5	5/5
A new derived feature is added	●	●	●		○	3/5	5/5
Random forest → gradient boosting		●	●	○		2/5	5/5
The response gets a new field				●	○	1/5	5/5
The decision threshold changes				●	○	1/5	5/5
The ML library is upgraded		●	●	●		3/5	5/5

HOW TO READ THE MATRIX

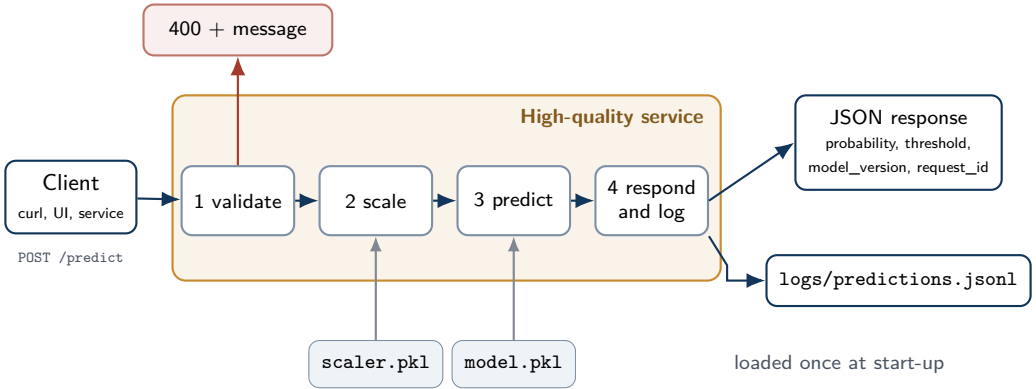
● **Must change:** the module has to be edited ○ **Check or retest:** no edit expected, but verify

n/5: how many of the five components must change. *Illustrative: preprocessing lives inside the artifact.*

Low coupling keeps each change inside one to three modules; the artifact is the boundary.

NEW **Feature:** an input variable. **Threshold:** probability above which we decide "fraud". **Gradient boosting:** many small trees added one by one. **Entangled:** any change touches every part.

Example: a prediction service as a pipeline of responsibilities



NEW **Endpoint:** one address of a service: method plus path, e.g. `POST /predict`. **JSON:** a text format for structured data. **Scaler:** rescales numeric features exactly as in training. **Log:** a record of each request, kept for audit. **curl:** a command-line tool that sends an HTTP request.

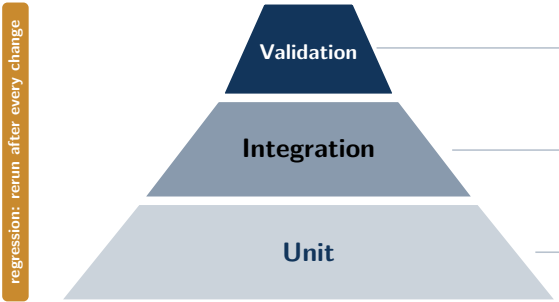
5 Quality assurance

Previously: separating what changes and agreeing on contracts keeps change cheap.

IN THIS PART

- Testing levels
- Testing ML systems: code, data, and models
- Evidence: the same six requests sent to two implementations

Testing levels



Does it meet the requirement?
Acceptance checks, e.g. a latency budget or deterministic output. Few, slow.

Components working together
A model artifact and a service endpoint, called as one. Fewer, slower.

Small pieces of logic
Feature functions, request parsing, input validation. Many, fast.

NEW **Unit test:** checks one small piece. **Integration test:** checks parts working together. **Acceptance test:** checks a requirement. **Regression:** a change breaks what used to work.

Testing ML systems: code, data, and models

Code	Data	Model
• Unit and integration tests, as in any software.	• Schema, ranges, and missing values are checked.	• Metrics on held-out data, overall and per slice.
• Does the service reject invalid input?	• Training and serving data are compared.	• Behavior under small input changes.
• Does the pipeline run end to end?	• A bad batch stops the pipeline early.	• Comparison with the previous model.

After Breck et al., The ML Test Score (2017).

NEW **Held-out data:** examples kept out of training and used only to evaluate. **Slice:** a subset of the data, e.g. one region. **Invariance test:** a small irrelevant change must not change the prediction.

Evidence: the same six requests sent to two implementations

	Low-quality service legacy, model path patched	**High-quality service** corrected counterpart
valid vector (30 numbers)	✓ **200** prediction	✓ **200** prediction
no "features" key	✗ **500** crash	✓ **400** rejected
29 values instead of 30	✗ **500** crash	✓ **400** rejected
text instead of a number	✗ **500** crash	✓ **400** rejected
NaN instead of a number	! **200** answered anyway	✓ **400** rejected
JSON list, not an object	✗ **500** crash	✓ **400** rejected
	invalid rejected: 0 of 5	**invalid rejected: 5 of 5**

Flask test client; the low-quality service ran with `streamlit/model.pkl` standing in for its missing pickle.

A crash is visible; a wrong answer is not:
the low-quality service answers 200 for NaN.

NEW **NaN:** "not a number": a missing or undefined numeric value. **Silent failure:** a wrong answer that looks fine. **Crash:** the service fails visibly, with HTTP 500. **Test client:** calls the service inside a test, without a network.

6 Evolution and MLOps

Previously: tests tell us whether the system still works as it changes.

IN THIS PART

- Maintenance and evolution: four kinds of change
- Software costs more to keep than to build
- Reproducibility needs configuration management
- From DevOps to MLOps: two loops
- From notebook to production: a ladder of discipline

Maintenance and evolution: four kinds of change

Corrective

Fix defects found in use.

In ML systems: a feature function that mishandles missing values.

Adaptive

Keep working as the environment changes.

In ML systems: retraining on new data, a library upgrade, a schema change.

Perfective

Add or improve capabilities.

In ML systems: a new feature, a better model, a lower latency.

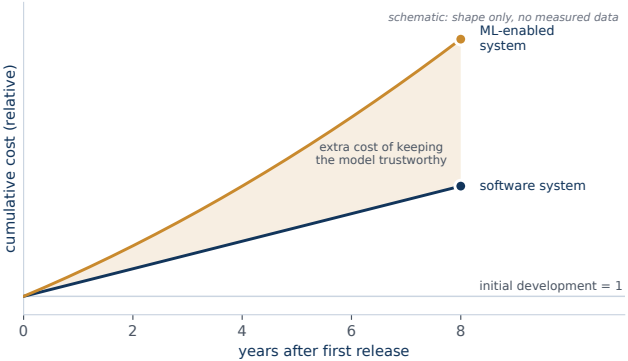
Preventive

Reduce the cost of future change.

In ML systems: refactoring, tests, documentation, pipeline cleanup.

NEW **Retraining:** fitting the model again on newer data. **Schema change:** a change in the structure of the data. **Refactoring:** restructuring code without changing what it does.

Software costs more to keep than to build

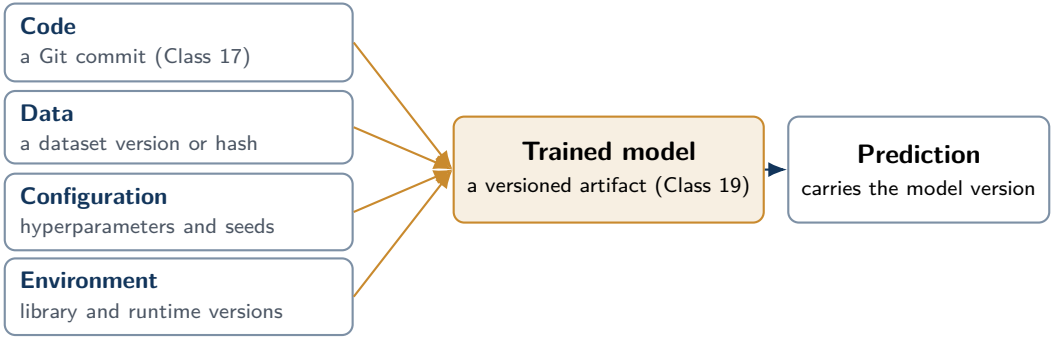


Sources: Sommerville, *Software Engineering*; Glass (2002), Fact 41: maintenance is 40–80% of software cost; Sculley et al. (2015), for ML systems.

- Software often costs more to maintain than to develop.
- Requirements change as users learn what they really need.
- In ML systems, data work, monitoring, and retraining push the maintenance share even higher.

NEW **Maintenance:** all work on a system after its first release. **Cumulative cost:** total cost accumulated over time.

Reproducibility needs configuration management

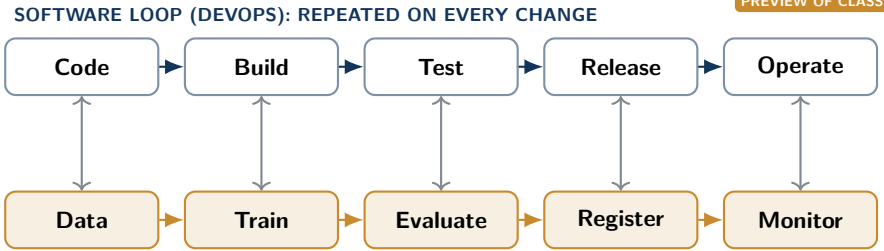


- **Reproducible:** the same four inputs give the same model.
- **Traceable:** any prediction can be followed back to all four. Experiment tracking (Class 19) records them together.

NEW **Configuration management:** versioning everything that defines a result. **Seed:** a number that makes randomness repeatable. **Experiment tracking:** a record of each training run. **Hash:** a short fingerprint of a file, used as a version.

From DevOps to MLOps: two loops

PREVIEW OF CLASS 26

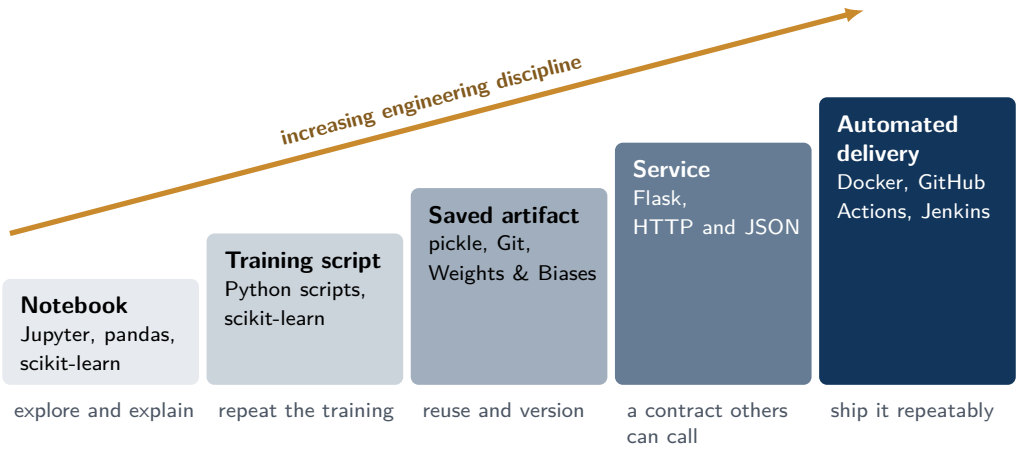


CI/CD builds, tests, and releases every change automatically; for ML, the data and the model artifact are checked as well.

MLOps runs both loops together, automated and observable.

NEW **DevOps:** development and operations working as one loop to build, test, release, and run software. **MLOps:** DevOps plus the model lifecycle: data, training, evaluation, registry, monitoring. **CI/CD:** automatic build, test, and release. **Drift:** data changes and the model degrades.

From notebook to production: a ladder of discipline



NEW **Docker:** packages a program with its environment so it runs the same anywhere. **GitHub Actions, Jenkins:** tools that run automated pipelines on every change.

Papers and materials

Papers and materials

Sommerville, Software Engineering; Glass, Facts and Fallacies of Software Engineering (2002); Winters, Manshreck, and Wright, Software Engineering at Google; Parnas, On the Criteria to Be Used in Decomposing Systems into Modules (1972); Sculley et al., Hidden Technical Debt in Machine Learning Systems; Breck et al., The ML Test Score; Nahar et al., Collaboration Challenges in Building ML-Enabled Systems; Chip Huyen, Designing Machine Learning Systems; Martin Fowler, trade-offs in architecture.

From This Class to the Next

What stays with us

- Model quality and software quality are related but not interchangeable.
- Boundaries, tests, and versioned artifacts make an ML idea a dependable system.
- MLOps automates today's loops; it does not replace the engineering behind them.

Project use

Review your project with the same lenses: requirements, boundaries, tests, versioning, monitoring.

Next connection

Class 21 is a guest session (Cybertec). Class 22 turns needs and quality attributes into testable requirements.

Course thread: Engineering discipline around code and process sets up the same discipline around requirements: knowing what "good" means before we build.

Backup: design smells in ML projects

Smell	Principle at stake	Refactoring
A bare list of numbers whose order only the author knows	Implicit contract	Named fields, validated at the boundary
The decision threshold is implicit: whatever `predict()` does	Configuration mixed with logic	An explicit, documented setting
One function loads data, cleans it, trains, plots, and saves	Low cohesion	One responsibility per step, composed in a pipeline
The service imports code from the training notebook	Coupling to internals	Share only the artifact and its schema
Notebook cells depend on variables defined many cells earlier	Hidden dependency	Functions with explicit inputs and outputs
Retraining is a manual sequence of notebook cells	No automation boundary	A script with explicit inputs and outputs that a scheduler can run

In the high-quality service: an explicit threshold (`FRAUD_THRESHOLD`) and input validated at the boundary.

NEW **Smell:** a sign in the code that hints at a deeper design problem. **Boundary:** the point where data enters a module.